

1. Write a [C](#) program to print all prime numbers between two limits.

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int start, end, i, j, isPrime;
5
6     printf("Enter the starting limit: ");
7     scanf("%d", &start);
8
9     printf("Enter the ending limit: ");
10    scanf("%d", &end);
11
12    printf("Prime numbers between %d and %d are:\n", start, end);
13
14    for(i = start; i <= end; i++) {
15        if(i <= 1)
16            continue;
17
18        isPrime = 1; // assume number is prime
19
20        for(j = 2; j <= i / 2; j++) {
21            if(i % j == 0) {
22                isPrime = 0;
23                break;
24            }
25        }
26
27        if(isPrime == 1) {
28            printf("%d ", i);
29        }
30    }
31
32    return 0;
33 }
```

Output

```
Enter the starting limit: 2
Enter the ending limit: 5
Prime numbers between 2 and 5 are:
2 3 5

=== Code Execution Successful ===
```

2. Write a [C](#) program to count the number of digits in a number

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int num, count = 0;
5
6     printf("Enter a number: ");
7     scanf("%d", &num);
8
9     // Handle 0 separately
10    if (num == 0) {
11        count = 1;
12    } else {
13        // Convert negative number to positive
14        if (num < 0) {
15            num = -num;
16        }
17
18        while (num > 0) {
19            num = num / 10;
20            count++;
21        }
22    }
23
24    printf("Number of digits = %d", count);
25
26    return 0;
27 }
```

Output

```
Enter a number: 23
Number of digits = 2

=== Code Execution Successful ===
```

3. Write a [C](#) program to print the alphabet S in n x n matrix.

main.c	Output
<pre> 1 #include <stdio.h> 2 3 int main() { 4 int n, i, j; 5 6 printf("Enter size (n): "); 7 scanf("%d", &n); 8 9 for(i = 0; i < n; i++) { 10 for(j = 0; j < n; j++) { 11 12 if (i == 0 i == n/2 i == n-1) { 13 printf("*"); 14 } 15 else if (i < n/2 && j == 0) { 16 printf("*"); 17 } 18 else if (i > n/2 && j == n-1) { 19 printf("*"); 20 } 21 else { 22 printf(" "); 23 } 24 } 25 printf("\n"); 26 } 27 28 return 0; 29 } 30 31 </pre>	<pre> Enter size (n): 3 *** *** *** === Code Execution Successful === </pre>

4. Write a [C](#) program to print the pyramid [pattern](#).

```

*

***

*****

*****

```

main.c	Output
<pre> 1 #include <stdio.h> 2 3 int main() { 4 int n, i, j; 5 6 printf("Enter number of rows: "); 7 scanf("%d", &n); 8 9 for(i = 1; i <= n; i++) { 10 11 // Print spaces 12 for(j = 1; j <= n - i; j++) { 13 printf(" "); 14 } 15 16 // Print stars 17 for(j = 1; j <= (2 * i - 1); j++) { 18 printf("*"); 19 } 20 21 printf("\n"); 22 } 23 24 return 0; 25 } </pre>	<pre> Enter number of rows: 5 * *** ***** ***** ***** === Code Execution Successful === </pre>

5. Write a [C](#) program to find GCD of two numbers using loop.

main.c		Run	Output
1	#include <stdio.h>		Enter two numbers: 03
2			23
3	int main() {		GCD of 3 and 23 = 1
4	int a, b, i, gcd;		
5			=== Code Execution Successful ===
6	printf("Enter two numbers: ");		
7	scanf("%d %d", &a, &b);		
8			
9	// Find minimum of a and b		
10	int min = (a < b) ? a : b;		
11			
12	for(i = 1; i <= min; i++) {		
13	if(a % i == 0 && b % i == 0) {		
14	gcd = i;		
15	}		
16	}		
17			
18	printf("GCD of %d and %d = %d", a, b, gcd);		
19			
20	return 0;		
21	}		